YT²uTor - Physical ERD

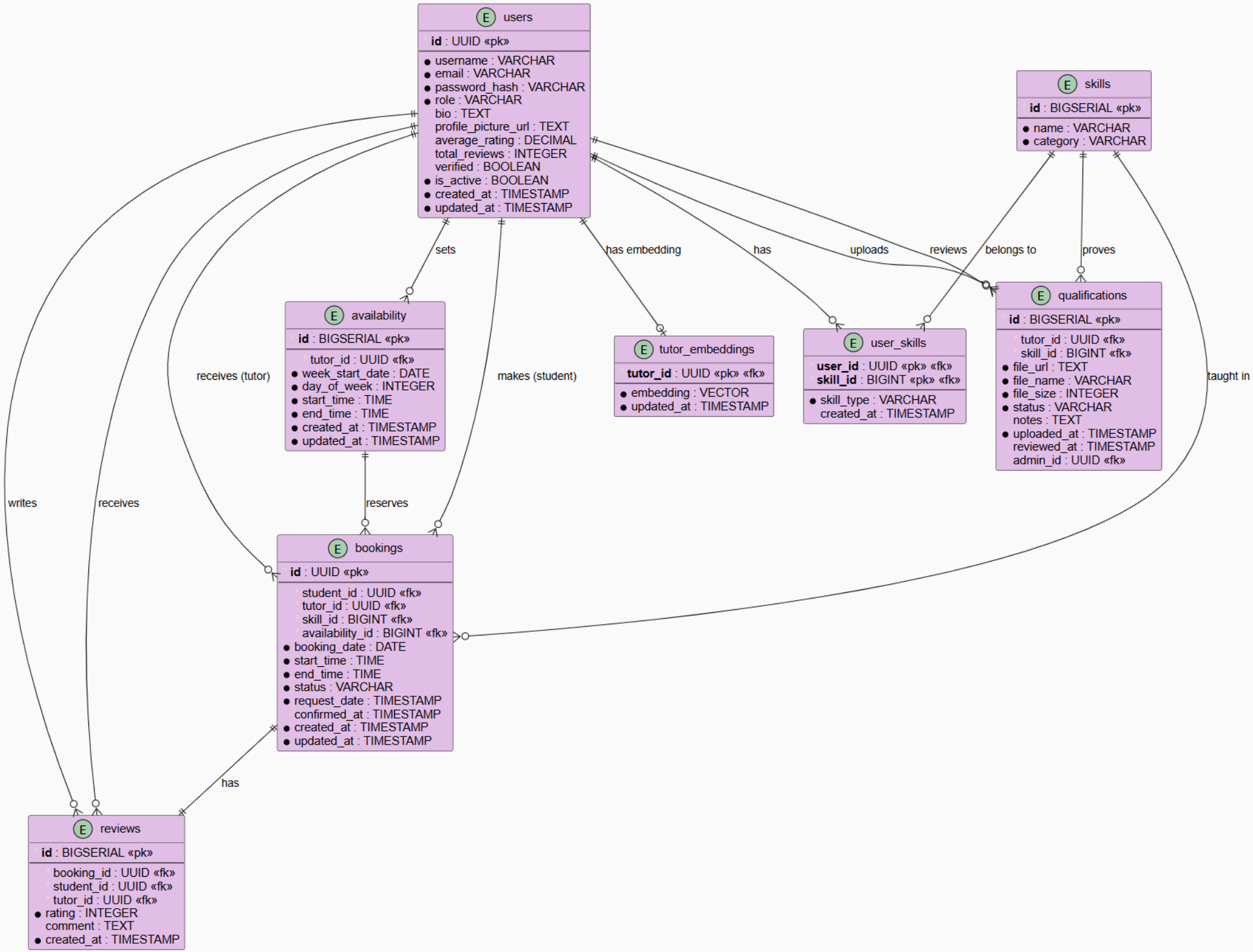

Table 1: Users

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier (UUID v7 for NFR-04)
username	VARCHAR(150)	UNIQUE, NOT NULL	Username for login
email	VARCHAR(255)	UNIQUE, NOT NULL	Email for login
password_hash	VARCHAR(255)	NOT NULL	Hashed password (PBKDF2)
role	VARCHAR(20)	NOT NULL, DEFAULT 'student', CHECK (role IN ('student', 'tutor', 'admin'))	User type
bio	TEXT	NULL	Bio (required for tutors, optional for students – enforced by application)
profile_picture_url	TEXT	NULL	Profile picture URL
average_rating	DECIMAL(3,2)	NULL	Average rating from reviews (only for tutors)
total_reviews	INTEGER	NULL	Total number of reviews received (only for tutors)
verified	BOOLEAN	NULL, DEFAULT FALSE	Whether tutor has been verified by admin (only for tutors)
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Account active status
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Registration date
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last profile update

Indexes:

- idx_users_email on email, idx_users_role on role

Note: *average_rating* and *total_reviews* are denormalized. Maintained by *ReviewService*.

Table 2: Skills

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique skill identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Skill name (e.g., "Python", "Baccalaureate Math")
category	VARCHAR(100)	NOT NULL	Skill category (e.g., "Programming", "Mathematics")

Indexes:

- idx_skills_name on name
- idx_skills_category on category

Note: *UUID v7 vs BIGSERIAL*
UUID v7 for *users.id and bookings.id* – time-ordered (good for indexes) + unguessable (security for URLs)
BIGSERIAL for all other IDs – faster, smaller storage, and used where the id is never exposed to users in our system

Table 3: User_Skills (Junction Table)

Column	Type	Constraints	Description
user_id	UUID	PRIMARY KEY, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	User identifier
skill_id	BIGINT	PRIMARY KEY, FOREIGN KEY REFERENCES skills(id) ON DELETE CASCADE	Skill identifier
skill_type	VARCHAR(20)	NOT NULL, CHECK (skill_type IN ('teaches', 'wants_to_learn'))	For tutors: 'teaches', for students: 'wants_to_learn'
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the skill was added to the user's profile

Indexes:

- idx_user_skills_user on user_id
- idx_user_skills_skill on skill_id
- idx_user_skills_type on skill_type

Note: Application validates that *skill_type* matches *user role*.

Table 4: Qualifications

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique qualification identifier
tutor_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	Tutor who uploaded the qualification
skill_id	BIGINT	NOT NULL, FOREIGN KEY REFERENCES skills(id) ON DELETE CASCADE	Skill that this qualification proves
file_url	TEXT	NOT NULL	Path to the uploaded file
file_name	VARCHAR(255)	NOT NULL	Original file name
file_size	INTEGER	NOT NULL, CHECK (file_size <= 5242880)	File size in bytes (max 5MB for NFR-07)
status	VARCHAR(20)	NOT NULL, DEFAULT 'pending', CHECK (status IN ('pending', 'approved', 'rejected'))	Verification status (pending, approved, or rejected)
notes	TEXT	NULL	Admin notes (e.g., rejection reason)
uploaded_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the qualification was uploaded
reviewed_at	TIMESTAMP	NULL	When the admin reviewed the qualification
admin_id	UUID	NULL, FOREIGN KEY REFERENCES users(id) ON DELETE SET NULL	Admin who reviewed the qualification (must have role='admin')

Indexes:

- idx_qualifications_tutor on tutor_id
- idx_qualifications_skill on skill_id
- idx_qualifications_status on status

Table 5: Availability

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique availability slot identifier
tutor_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	Tutor offering the availability slot
week_start_date	DATE	NOT NULL	Day of the week this availability applies to
day_of_week	INTEGER	NOT NULL, CHECK (day_of_week BETWEEN 0 AND 6)	Day of week (0 = Sunday, 6 = Saturday)
start_time	TIME	NOT NULL	Start time of the availability slot
end_time	TIME	NOT NULL, CHECK (end_time > start_time)	End time of the availability slot
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the availability slot was created
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last time the availability slot was updated

Indexes:

- idx_availability_tutor on tutor_id
- idx_availability_day_time on day_of_week, start_time

Note:NO *is_booked* column. **Availability** is derived from bookings table (confirmed bookings mark a slot as taken).

Table 6: Bookings

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY	Unique booking identifier
student_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE RESTRICT	Student making the booking
tutor_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE RESTRICT	Tutor being booked
skill_id	BIGINT	NOT NULL, FOREIGN KEY REFERENCES skills(id) ON DELETE RESTRICT	Skill being taught in the session
availability_id	BIGINT	NOT NULL, FOREIGN KEY REFERENCES availability(id) ON DELETE RESTRICT	Which availability slot was booked
booking_date	DATE	NOT NULL	Date of the session
start_time	TIME	NOT NULL	Start time of the session
end_time	TIME	NOT NULL, CHECK (end_time > start_time)	End time of the session
status	VARCHAR(20)	NOT NULL, DEFAULT 'pending', CHECK (status IN ('pending', 'confirmed', 'rejected', 'completed', 'cancelled'))	Current status of the booking
request_date	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the booking request was made
confirmed_at	TIMESTAMP	NULL	When the tutor confirmed or rejected the booking
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp

Indexes:

- idx_bookings_student on student_id
- idx_bookings_tutor on tutor_id
- idx_bookings_status_date on status, booking_date
- idx_bookings_availability on availability_id

Partial Unique Index:

- ux_booking_active_slot on availability_id WHERE status = 'confirmed' (prevents double-booking)

Note: *booking_date, start_time, end_time* are intentionally denormalized as a historical snapshot.

Table 7: Reviews

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique review identifier
booking_id	UUID	NOT NULL, UNIQUE, FOREIGN KEY REFERENCES bookings(id) ON DELETE CASCADE	Booking being reviewed (one review per booking)
student_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE RESTRICT	Student writing the review
tutor_id	UUID	NOT NULL, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	Tutor receiving the review
rating	INTEGER	NOT NULL, CHECK (rating BETWEEN 1 AND 5)	Star rating (1–5 stars)
comment	TEXT	NULL	Optional written feedback
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When the review was left

Indexes:

- idx_reviews_booking on booking_id
- idx_reviews_tutor on tutor_id
- idx_reviews_rating on rating
-

Note: Application constraint: review only allowed if *booking.status = 'completed'*.

Table 8: Tutor_Embeddings

Column	Type	Constraints	Description
tutor_id	UUID	PRIMARY KEY, FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	Tutor identifier (only for tutors with role='tutor')
embedding	VECTOR(384)	NOT NULL	Pre-computed embedding vector from Sentence Transformers for AI similarity search
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last time the embedding was recomputed (when tutor's skills change)

Indexes:

- idx_tutor_embeddings_tutor on tutor_id

Relationships Between Tables

From Table	To Table	Via Column	Relationship Type	Description
users	user_skills	user_id	One-to-Many	One user can have many skill entries
skills	user_skills	skill_id	One-to-Many	One skill can belong to many users
users	qualifications	tutor_id	One-to-Many	One tutor can upload many qualifications
skills	qualifications	skill_id	One-to-Many	One skill can have many qualifications
users	qualifications	admin_id	One-to-Many	One admin can review many qualifications
users	availability	tutor_id	One-to-Many	One tutor can set many availability slots
availability	bookings	availability_id	One-to-Many	One availability slot can have many booking attempts
users	bookings	student_id	One-to-Many	One student can make many bookings
users	bookings	tutor_id	One-to-Many	One tutor can receive many bookings
skills	bookings	skill_id	One-to-Many	One skill can be taught in many bookings
bookings	reviews	booking_id	One-to-One	One booking can have zero or one review
users	reviews	student_id	One-to-Many	One student can write many reviews
users	reviews	tutor_id	One-to-Many	One tutor can receive many reviews
users	tutor_embeddings	tutor_id	One-to-One	One tutor can have zero or one embedding vector